

SEN-107 Fundamental Data Structures
Assessment Activity SEN-107:00030, SEN-107:00040
Creating and Using Binary Sorted Trees
Creating and Using Trees with Arbitrary Branching Factors
Last update: 12 January 2025

Instructions

General Instructions

Write a program that solves the problem described below. Your program should read from the standard input and write to the standard output. We have provided example input and output files. When running your program on the example input file, it should produce the same output as in the example output file. While this example is a good starting point, you should test your program thoroughly, with multiple different inputs too. Your program should correctly handle all possible cases specified below. Correctness will be part of your assessment score. Please also follow the coding standards in *C_CodingStandards.pdf*. Violating the coding standards will lower your assessment score.

Submit your *C source code* only. Your source code must be a single plain text file. Do not submit code as a doc file, pdf file, IDE project, or anything else. We must be able to compile and run your code on our computer, without any warnings or errors.

For convenience, we have provided a template for you, which already contains the skeleton code that handles the input-output. You may implement your solution using any data structure you learn from lectures (or beyond). However, you are **not** allowed to use other external libraries, such as C++ Standard Template Library (STL).

Assignment-Specific Instructions

For this assessment, you should submit a single file called **orgchart.c**. As stated above, we have already provided a template for this file. The example input and output files are **orgchart.in** and **orgchart.out**. Ensure your program works correctly with this case. Also test your program with a variety of edge cases, such as deeply nested organization trees, invalid supervisor references, and invalid queries.

Problem Statement: Building and Searching an Organization Chart

The command-and-control structure of organizations, such as companies or government agencies, is often hierarchical. An organization chart is a tree-based representation of such a control structure,

where people higher in the chart supervise or control the people below them. The root of an organization tree is the person responsible for the entire organization, such as the CEO or director.

Your task is to write a program that reads input from the standard input/output describing people and relationships in an organization, builds tree(s) to represent these relationships, and performs several operations:

- **Build the Organization Chart:**
 - Parse the input and construct the organization chart tree. Each person is a tree node, and the supervisor acts as the parent.
 - Input lines may appear in any order, but a supervisor must be defined before their subordinates.
 - If a supervisor is not found, print an error message and skip the entry.
- **Output the Organization Chart:**
 - Perform a pre-order traversal of the tree and print the contents using three dots to represent one level hierarchy.
- **Search for an Employee:**
 - Allow the user to input a name and Employee ID to search for an employee.
 - Print the job title, supervisor's name, and Employee ID. If the person is the CEO, indicate they are at the top of the chart. If the employee is not found, print an error message.
- **Print a Sorted List:**
 - Print all employees sorted by name, and in case of duplicates, by Employee ID.

Input Description

The input will consist of the following:

- An integer N representing the number of employees.
- An integer Q representing the number of queries.
- N lines of employee data in the format: [Name] [EmployeeId] [JobTitle] [SupervisorName] [SupervisorEmployeeId]
 - SupervisorName and SupervisorEmployeeId are "--" for the root.
- Q lines of queries in the format: [Name] [EmployeeId]

You can assume that there will be no duplicate entries where both the name and Employee ID are the same.

Input values are space-separated. Ensure no extra spaces are present at the beginning or end of the lines.

Output Description

- If a supervisor is not found during the add operation, print an error message and skip the entry e.g. `cannot_add Sam 1333288`.
- Print the organization chart in pre-order traversal with each level of hierarchy represented by three dots (e.g., '...').
- For each query, print the employee's details or an error message. If the employee is not found, print `not_found`.
- After the queries, print the sorted list (by name and then Employee ID) of all employees.

Note

- There is no limit on the number of children that a tree node in the organization chart may have.

- This assessment tests your understanding of both binary sorted and general trees. Therefore, to get credit for both skills, your program must build two trees: the organization chart, and a sorted binary tree to use as an index in finding employees. The sorted binary tree should use the combination of the Name and the EmployeeId as the key for sorting. You can concatenate the two strings for this purpose (with name first). The sorted binary tree nodes should point to the nodes in the main organization chart. Do not duplicate all the information (e.g. job title) in the binary tree.

Example Input & Output

Input	Output
<pre> 8 3 Julie 1012345 CEO -- -- Mark 1012365 VP_Finance Julie 1012345 Janet 1012234 Chief_Accountant Mark 1012365 Lisa 1016541 VP_Technology Julie 1012345 Howard 1329870 Auditor Mark 1012365 Roger 1065422 SWE_Manager Lisa 1016541 Tania 1238966 IT_Director Lisa 1016541 Maria 1899877 VP_Marketing Julie 1012345 Janet 1012234 Roger 1065422 Maria 1899877 </pre>	<pre> Julie 1012345 CEO ...Mark 1012365 VP_FinanceJanet 1012234 Chief_AccountantHoward 1329870 Auditor ...Lisa 1016541 VP_TechnologyRoger 1065422 SWE_ManagerTania 1238966 IT_Director ...Maria 1899877 VP_Marketing Chief_Accountant Mark 1012365 SWE_Manager Lisa 1016541 VP_Marketing Julie 1012345 Howard 1329870 Auditor Janet 1012234 Chief_Accountant Julie 1012345 CEO Lisa 1016541 VP_Technology Maria 1899877 VP_Marketing Mark 1012365 VP_Finance Roger 1065422 SWE_Manager Tania 1238966 IT_Director </pre>

Explanation: 8 is the number of employees in this organization to be added and 3 is the number of queries. The employee information begins from Julie and ends at Maria. Then, we query for information about Janet, Roger, and Maria. The output consists of three sections. First is the organization chart in pre-order traversal, using three dots to show hierarchy e.g. Julie's subordinates are Mark, Lisa, and Maria. The second section is an answer to the employee search queries, containing [[JobTitle] [SupervisorName] [SupervisorEmployeeId]] e.g. Chief_Accountant Mark 1012365 for query Janet 1012234. The last part is the sorted list of employees by name and then ID.

